# FLOODCAST

Predict the Surge Before It Strikes

**Disaster Risk & Early Warning Systems** Regression Challenge 24-Hour Hackathon · November 2026

## Overview

Every year, floods kill thousands and displace millions. The difference between a community that evacuates in time and one that doesn't often comes down to a single number: *how severe will the river surge be?*

Meteorological agencies have the data. What they lack are models that generalize — systems that can estimate flood severity at a river station they have never seen before, using only rainfall, temperature, and basic catchment characteristics.

**That is what you are building.**

## Problem Statement

Given historical river discharge, rainfall, and temperature data for a set of river stations, build a regression model that predicts the **peak daily discharge (in cubic feet per second)** at a station for each day in the test period.

**Formally:**

For each **(station_id, date)** pair in **test.csv**, predict:

- discharge_cfs — river discharge in cubic feet per second (continuous, non-negative)

**What discharge means:**

Discharge is the volume of water flowing past a river cross-section per second. Low discharge = normal river. High discharge = flood risk. Very high discharge = dangerous flooding. The model must predict the actual number, not a category.

## Dataset

You will use **three real, publicly available datasets**. All are free. Direct download links and exact instructions are below. We will also provide pre-processed versions of the core files at the event start.

### Dataset 1 — USGS National Water Information System (Primary)

This is your primary target data source. It contains daily discharge measurements from thousands of US river gauge stations going back 50+ years.

| Field | Details |
|---|---|
| What you get | Daily river discharge (cfs) per station, with timestamps and quality flags |
| Coverage | Thousands of stations across the United States, records from 1900s to present |
| Direct link | `https://waterdata.usgs.gov/nwis/dv` |
| How to download | Go to link above → Select state → Select stations → Download as Tab-separated or CSV → Select parameter 00060 (discharge) |
| API (fastest method) | `https://waterservices.usgs.gov/nwis/dv/?format=json&sites;=SITE_ID&startDT;=2010-01-01&endDT;=2023-12-31&parameterCd;=00060` |
| Key columns | site_no, datetime, discharge_cfs (00060_Mean), quality_cd |
| Cost | Free. No registration. |

*Recommended stations to start with: 01646500 (Potomac, DC), 07010000 (Mississippi, St. Louis), 08158000 (Colorado, Austin TX), 02087500 (Neuse River, NC), 11447650 (Sacramento River, CA)*

### Dataset 2 — Open-Meteo Historical Weather API (Rainfall + Temperature)

This provides the meteorological inputs your model needs to predict discharge. Match weather data to each USGS station by latitude and longitude.

| Field | Details |
|---|---|
| What you get | Daily precipitation (mm), max/min temperature (°C), wind speed, soil moisture — per lat/lon coordinate |
| Coverage | Global. Any lat/lon. Historical records from 1940 to present. |
| Direct link | `https://open-meteo.com/en/docs/historical-weather-api` |
| API example | `https://archive-api.open-meteo.com/v1/archive?latitude=38.9&longitude;=-77.0&start;_date=2010-01-01&end;_date=2023-12-31&daily;=precipitation_sum,temperature_2m_max,temperature_2m_min,soil_moisture_0_to_7cm&timezone;=America%2FNew_York` |
| Key columns | time, precipitation_sum, temperature_2m_max, temperature_2m_min, soil_moisture_0_to_7cm |
| Cost | Free. No API key needed. |

### Dataset 3 — USGS StreamStats Catchment Attributes (Static Features)

These are the physical characteristics of each river catchment — drainage area, slope, land cover, elevation. These are static features that help your model understand why different stations behave differently.

| Field | Details |
|---|---|
| What you get | Drainage area (sq mi), mean basin elevation, mean slope, % forest cover, % impervious surface per station |
| Direct link | `https://streamstats.usgs.gov/ss/` |
| How to access | Go to link → Click a station on the map → 'Basin Characteristics' → Download as JSON or use the API: https://streamstats.usgs.gov/streamstatsservices/ |
| API example | `https://streamstats.usgs.gov/streamstatsservices/characteristics.json?rcode=MD&workspaceID;=WORKSPACE_ID&includeWatershed;=true` |
| Key columns | DRNAREA (drainage area), ELEV (mean elevation), FOREST (% forest), SLOPE_PCT (mean slope) |
| Cost | Free. No registration. |

**How the Three Datasets Join Together:**

| Join Key | Dataset A | Dataset B | Result |
|---|---|---|---|
| station_id | USGS discharge | StreamStats attributes | Station + its physical characteristics |
| lat, lon, date | USGS discharge | Open-Meteo weather | Daily discharge + same-day weather |
| All three | Combined | All features | Final model-ready dataset: discharge, weather, catchment |

*We will provide a pre-joined starter CSV at the event with 5 stations already merged. You are free to extend it with more stations from the same sources.*

## Input / Output Contract

**Files you will receive:**

| File | Contents |
|---|---|
| train.csv | station_id, date, discharge_cfs, precipitation_mm, temp_max_c, temp_min_c, soil_moisture, drainage_area_sqmi, mean_elevation_ft, slope_pct, forest_cover_pct |
| test.csv | station_id, date, precipitation_mm, temp_max_c, temp_min_c, soil_moisture, drainage_area_sqmi, mean_elevation_ft, slope_pct, forest_cover_pct (NO discharge_cfs — this is what you predict) |
| sample_submission.csv | station_id, date, discharge_cfs (filled with zeros — replace with your predictions) |
| station_metadata.csv | station_id, station_name, latitude, longitude, state, river_name |

**Required output — predictions.csv:**

Your submission must be a CSV file with **exactly these three columns** and one row per test record:

```
station_id , date , discharge_cfs
```

*discharge_cfs must be a non-negative float. Any NaN, negative value, missing row, or extra column will cause an automatic scoring error.*

**Validation rules — your file will be rejected if:**

- Any station_id or date in test.csv is missing from your predictions
- Any predicted discharge_cfs value is negative or NaN
- The file has more or fewer rows than test.csv
- Column names are misspelled or in wrong order
- File is not UTF-8 encoded CSV

## Evaluation Criteria

Your submission is auto-scored against hidden ground-truth discharge values. The final leaderboard score is a weighted combination of four technical metrics. **There is no manual component in the auto-score.**

### Primary Metric — Root Mean Squared Error (RMSE)

```
RMSE = sqrt( mean( (predicted_discharge - actual_discharge)^2 ) )

Lower RMSE is better. RMSE is computed across all rows in test.csv. The
leaderboard ranks teams by RMSE ascending.
```

*RMSE penalizes large errors heavily. A model that mostly predicts correctly but catastrophically fails on flood peaks will score worse than a model that is consistently slightly off. This is intentional — missing a flood peak matters most.*

### Full Scoring Breakdown

| Metric | Weight | What it measures | Direction |
|---|---|---|---|
| RMSE (overall) | **40%** | Overall prediction accuracy across all stations and dates | *Lower = better* |
| RMSE on extreme events (top 10% discharge days) | **30%** | Accuracy specifically on high-discharge days — the most dangerous and hardest to predict | *Lower = better* |
| MAE (Mean Absolute Error) | **20%** | Average absolute error — less sensitive to outliers, measures general reliability | *Lower = better* |
| R-squared (R2) | **10%** | Proportion of variance explained — measures how well the model captures discharge patterns | *Higher = better* |

**How the final score is computed:**

Each metric is normalized to a 0–100 scale using the best and worst scores seen across all submissions. The weighted average of the four normalized scores gives the final leaderboard score (higher = better).

```
RMSE_norm = 100 * (worst_RMSE - your_RMSE) / (worst_RMSE - best_RMSE)
```

*The normalization is computed after the submission window closes using the actual range of scores. This means your leaderboard position during the event is based on raw RMSE only. Final ranks use the weighted composite.*

### What is judged — explicit breakdown:

| What is measured | How | Hardcoded check? |
|---|---|---|
| Overall discharge prediction accuracy | RMSE on all test rows | **Yes — auto** |
| Flood peak accuracy (extreme events) | RMSE on rows where actual discharge is in top 10% per station | **Yes — auto** |

| | | |
|---|---|---|
| No negative predictions | Any row with discharge_cfs < 0 is auto-flagged and penalized | **Yes — hard reject** |
| No missing predictions | Every (station_id, date) in test.csv must have a prediction | **Yes — hard reject** |
| No data leakage | Test dates are strictly after training dates. Using test targets voids submission. | **Yes — disqualified** |
| General prediction reliability (MAE) | MAE on all test rows | **Yes — auto** |
| Pattern capture quality (R2) | R2 score on all test rows | **Yes — auto** |

**Hard disqualification rules (not scored — submission voided):**

1. Negative discharge values in predictions
2. Missing rows in predictions.csv
3. Submitting actual test labels as predictions (leakage)
4. Submitting a predictions.csv that was not generated by running your code

## Submission Requirements

| Item | What to include | Required |
|---|---|---|
| GitHub Repo | Public repo with all code, README, requirements.txt or environment.yml | **Yes** |
| predictions.csv | Your model's output on test.csv — columns: station_id, date, discharge_cfs | **Yes** |
| README.md | Team name, members, approach summary (3–5 sentences), how to run the code | **Yes** |
| model_card.md | Which features used, model type, train/val RMSE, known limitations | **Yes** |
| Presentation (3–5 slides) | Problem → Data → Model → Results → Limitations. PDF or link. | **Yes** |

**GitHub repo structure (minimum required):**

```
floodcast-team-name/
  ■■ README.md
  ■■ requirements.txt
  ■■ solution.ipynb  or  solution.py
  ■■ predictions.csv
  ■■ model_card.md
```

**Submission steps:**

1. Fork or create a public GitHub repo with the structure above

2. Run your model on test.csv and save output as predictions.csv
3. Push all files to GitHub
4. Submit your GitHub repo link + predictions.csv via the Google Form shared at the event
5. Deadline: Hour 20 — no submissions accepted after this

## Rules

**Allowed:**

- Any ML framework — scikit-learn, XGBoost, LightGBM, PyTorch, TensorFlow
- Pre-trained models as a starting point, provided they are declared in model_card.md
- Adding more stations from USGS beyond the provided starter data
- Any feature engineering on the provided columns
- Teams of 1 to 4 members

**Not allowed:**

- Using test.csv discharge values as model input (data leakage)
- Manually editing predictions.csv without running a model
- Submitting a pre-built model from before the event
- Sharing predictions or code with other teams

## Event Timeline

| Time | Event |
|---|---|
| Hour 0 | PS released, datasets shared, starter CSV distributed |
| Hour 1 | Starter notebook walkthrough by organizers |
| Hour 6 | Optional organizer check-in |
| Hour 20 | **Submission deadline — GitHub link + predictions.csv due** |
| Hour 22 | 3-minute presentations begin |
| Hour 24 | Winners announced |